

From Static Website to Interactive Website

What We Learned So Far

HTML gives structure.

```
h1{ color:red; }
```

CSS gives styling.

```
alert("Welcome");
```

JavaScript gives behavior.

⚠ **Problem:** Can users interact with the webpage?

Example:

- Click Button → Change Text
- Add To Cart → Update Count
- Submit Form → Show Message

How? Using: DOM Manipulation + Events

Real World Example — Instagram:

- User Clicks Like
- ↓ Event Triggered
- ↓ Like Count Updated
- ↓ DOM Changed

Important Concept — Modern Frontend Development =

- User Action
- ↓ JavaScript Logic
- ↓ DOM Update

Understanding DOM Manipulation

Concept: DOM Manipulation means changing webpage content, structure, or styling using JavaScript.

Three Things JavaScript Can Change:



Content

Hello → Welcome Student

Styles

color:red; → color:blue;

Structure

Add new elements dynamically

Industry Usage — Used in:

Dashboards

Ecommerce Sites

Social Media

Banking Applications

innerHTML Deep Dive

Concept: `innerHTML` changes entire HTML inside an element.

HTML:

```
<div id="box">Welcome</div>
```

JavaScript:

```
document.getElementById("box")  
  .innerHTML = "<h1>Hello Students</h1>";
```

Browser Result:

```
<h1>Hello Students</h1>
```

Why Companies Use innerHTML

When API data arrives:

```
let product = {  
  name: "Laptop",  
  price: 50000  
};
```

Generate UI dynamically:

```
box.innerHTML = `  
  <h2>${product.name}</h2>  
  <p>${product.price}</p>  
`;
```

⚠️ **Warning:** Never blindly insert user input. Can create security issues — **XSS Attack** (only awareness, no deep dive).

textContent vs innerHTML

Interview Question: What is the difference between `innerHTML` and `textContent`?

innerHTML

```
element.innerHTML = "<h1>Hello</h1>";
```

Creates an actual heading element.

Why Companies Prefer textContent?

- Safer
- Faster
- No HTML parsing

textContent

```
element.textContent = "<h1>Hello</h1>";
```

Displays the literal text: `<h1>Hello</h1>`

Best Practice

Need HTML?

↓ innerHTML

Need Text?

↓ textContent

Real Usage — Notification Systems:

```
message.textContent = "Payment Successful";
```

Style Manipulation

Concept: JavaScript can directly modify CSS properties.

Example:

```
let title = document.getElementById("title");  
title.style.color = "red";
```

Multiple Properties:

```
title.style.color = "white";  
title.style.backgroundColor = "black";  
title.style.padding = "20px";
```

⊗ **Problem:** Imagine changing 20 properties. Very difficult.

Industry Practice: Avoid excessive inline styling. Prefer CSS Classes.

Important Interview Point:

style.property

↓ Inline Styles

classList

↓ Professional Approach



classList Deep Dive

Why classList Exists? Instead of writing many inline styles:

```
title.style.color = "white";  
title.style.backgroundColor = "black";
```

Use a CSS class:

```
.dark {  
  background: black;  
  color: white;  
}
```

JavaScript:

```
document.body.classList.add("dark");
```



Add Class

```
element.classList.add("active");
```



Remove Class

```
element.classList.remove("active");
```



Toggle Class

```
element.classList.toggle("active");
```



Check Class

```
element.classList.contains("active");
```

Real World Usage: Dark Mode, Sidebar Open, Dropdown Open, Selected Menu, Active Button

Events Deep Dive

Concept: Event = User Action

Common Events

Event	Trigger
click	Mouse Click
input	Typing
submit	Form Submit
change	Select Change
mouseover	Hover
keyup	Keyboard Release

Real World Flow

01

User Action

02

Event Triggered

03

JavaScript Function Runs

04

DOM Updates

Example:

```
button.onclick = function(){
  alert("Clicked");
}
```

Industry Reality: Most applications are event-driven. React is also event-driven.

addEventListener Deep Dive

Why Not onclick?

Old Way:

```
button.onclick = function(){ }
```

Modern Way:

```
button.addEventListener("click", function(){  
  console.log("Clicked");  
});
```

Advantage: Can attach multiple listeners.

```
button.addEventListener("click", updateCount);  
button.addEventListener("click", saveAnalytics);
```

Real World Example — Ecommerce

Add To Cart Click triggers:

Multiple tasks — multiple listeners on the same event.

→ Update Cart

→ Save Analytics

→ Show Notification

Real Project — Theme Changer

Industry Concepts Used

DOM Selection

Event Handling

CSS Classes

Dynamic UI

CSS:

```
.dark {  
  background: black;  
  color: white;  
}
```

JavaScript:

```
const button = document.getElementById("themeBtn");  
button.addEventListener("click", () => {  
  document.body.classList.toggle("dark");  
});
```

Concepts Learned: Select Element → Listen Event → Toggle Class → Update UI

Real Usage — Dark Mode: YouTube, LinkedIn, Instagram



Real Project — Counter App

Why Counter App?

Almost every application uses counters.

Likes	Followers
Cart Quantity	Notifications

HTML:

```
<h1 id="count">0</h1>
```

JavaScript:

```
let count = 0;
const display = document.getElementById("count");
document.getElementById("add")
  .addEventListener("click", () => {
    count++;
    display.textContent = count;
  });
```

Separation of Concerns — Good Practice:

HTML ↓ Structure	CSS ↓ Styles	JavaScript ↓ Behavior
---------------------	-----------------	--------------------------

DOM Manipulation Formula — Students should remember:

- 1 Select Element
- 2 Store Element
- 3 Add Event
- 4 Write Logic
- 5 Update DOM

Complete Learning Flow

01

User Click

02

Event Triggered

03

Variable Updated

04

DOM Updated

05

UI Updated



Event Object: event contains information about the user action.
`button.addEventListener("click", function(event){ console.log(event); });`